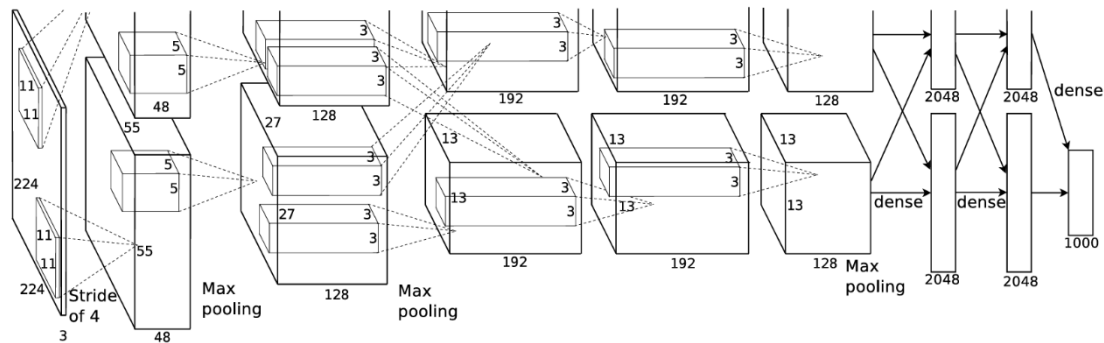


1. Config error



AlexNet

2. RuntimeError: AlexNet ON Cells.pt

> mat1 and mat2 shapes cannot be multiplied (32x3072 and 2048x1024)

The flattened spatial dimensions (4x4) multiplied by output channels (192) equals 3072. The classifier's input must match this size for the matrix multiplication to work.

Formula for Conv2d:

$$\text{output_size} = (\text{input_size} - \text{kernel_size} + 2 * \text{padding}) / \text{stride} + 1$$

Formula for MaxPool2d:

$$\text{output_size} = (\text{input_size} - \text{kernel_size} + 2 * \text{padding}) / \text{stride} + 1$$

Trace through AlexNet:

Start: 64x64

Layer 1: Conv2d(kernel=7, stride=2, padding=3)

$$= (64 - 7 + 2 * 3) / 2 + 1$$

$$= (64 - 7 + 6) / 2 + 1$$

$$= 63 / 2 + 1$$

$$= 31.5 + 1 = 32.5 \rightarrow 32$$

After Conv1: **32x32, channels=48**

Layer 2: MaxPool2d(kernel=3, stride=2, padding=1)

$$= (32 - 3 + 2 * 1) / 2 + 1$$

$$= 31 / 2 + 1$$

$$= 16.5 \rightarrow 16$$

After MaxPool1: **16x16, channels=48**

(Continue this for Conv2, MaxPool2, Conv3-5, MaxPool3...)

Final: After all layers → 4x4, channels=192

Flattened size calculation:

Flattened = height × width × channels
= 4 × 4 × 192
= 16 × 192
= 3072

Now — the BUG!

Look at AlexNet's classifier:

`nn.Linear(2048, 1024)`, # ← expects 2048, but gets 3072!

3. Output size:

> In classifier layer I changed the last layer output size hardcoded to generalise form see that in code.

4. RuntimeError:

> **0D or 1D target tensor expected, multi-target not supported: outputs shape: torch.Size([32, 2]), labels shape: torch.Size([32, 1])**

The thing is before I set the “NUM_CLASSES”=2, so at the end output of shape is [32,2]. when data goes into the BinaryCrossEntropy then still need to convert from [32,2] -> [32]. Require Squeezing. Now the question is where you should perform changes? In 1. Data.py or 2. Fit.py. Right ans is in fit.py.

5. IndexError: Target 6 is out of bounds.

After solving “4” number point when I ran the file then I got this error.

```
print("Unique labels:", torch.unique(labels)) : [0, 1, 2, 3, 4, 5, 6, 7]
print("Number of unique classes:", len(torch.unique(labels))) : 8
```

So there are total 8 unique classes present in data; so now in config.js file I changed “NUM_CLASSES”=8. This is the right number of output classes.

6. Model Accu. Low

Epoch [01/50] | Train Loss: 3.0231 - Train Acc: 18.29% | Val Loss: 2.0250 - Val Acc: 19.17%
Epoch [02/50] | Train Loss: 2.0180 - Train Acc: 19.22% | Val Loss: 2.0367 - Val Acc: 19.17%
Epoch [03/50] | Train Loss: 2.0271 - Train Acc: 18.51% | Val Loss: 2.0304 - Val Acc: 17.70%
Epoch [04/50] | Train Loss: 2.0375 - Train Acc: 19.02% | Val Loss: 7489.7112 - Val Acc: 13.46%

To improve the model Accuracy:

➤ ``self.optimizer.zero_grad()`` <- bcs of no zero grad; there is grad explode.

Without it, gradients **accumulate** across batches instead of being reset. So:

- Batch 1: gradients = [0.5, 0.3, ...]

- Batch 2: gradients = [0.5, 0.3, ...] + [0.2, 0.1, ...] = [0.7, 0.4, ...] ← accumulated!
- Batch 3: gradients keep growing → **explode!**

Still too low;

- Change **drop_rate** from **0.99** -> **0.5** [Means before the model turning of 99% of neurons but it's 50% of them are off]

Epoch [01/50] | Train Loss: 1.0747 - Train Acc: 58.23% | Val Loss: 2.6553 - Val Acc: 38.04%
 Epoch [02/50] | Train Loss: 0.5043 - Train Acc: 81.51% | Val Loss: 0.3890 - Val Acc: 85.66%
 Epoch [03/50] | Train Loss: 0.3579 - Train Acc: 87.46% | Val Loss: 0.5323 - Val Acc: 82.30%
 Epoch [04/50] | Train Loss: 0.2951 - Train Acc: 90.01% | Val Loss: 0.1731 - Val Acc: 94.15%
 Epoch [05/50] | Train Loss: 0.2689 - Train Acc: 91.09% | Val Loss: 0.9158 - Val Acc: 76.15%
 Epoch [06/50] | Train Loss: 0.2344 - Train Acc: 92.25% | Val Loss: 0.2887 - Val Acc: 90.86%
 Epoch [07/50] | Train Loss: 0.2285 - Train Acc: 92.55% | Val Loss: 0.4271 - Val Acc: 84.86%
 Epoch [08/50] | Train Loss: 0.2001 - Train Acc: 93.53% | Val Loss: 0.2040 - Val Acc: 92.61%
 Epoch [09/50] | Train Loss: 0.1701 - Train Acc: 94.19% | Val Loss: 0.2433 - Val Acc: 91.59%
 Epoch [10/50] | Train Loss: 0.1719 - Train Acc: 94.48% | Val Loss: 0.2716 - Val Acc: 91.08%

```
{
  "DATA": "cells.pt",
  "DATA_PATH": "./Data/",
  "BATCH_SIZE": 32,
  "LEARNING_RATE": 0.001,
  "EPOCHS": 50,
  "MODEL": "AlexNet",
  "CHANNELS": 3, # RGB
  "NUM_CLASSES": 8
}
```

2. Now same **AlexNet** model but Dataset: **chest.pt** change:

```
{
  "DATA": "chest.pt",
  "DATA_PATH": "./Data/",
  "BATCH_SIZE": 32,
  "LEARNING_RATE": 0.001,
  "EPOCHS": 50,
  "MODEL": "AlexNet",
  "CHANNELS": 3,
  "NUM_CLASSES": 2
}
```

About data:

Image shape: torch.Size([5232, 1, 64, 64])

Unique labels: tensor([0, 1])

Num classes: 2

- **RuntimeError: Given groups=1, weight of size [48, 3, 7, 7], expected input[32, 1, 64, 64] to have 3 channels, but got 1 channels instead**

Channel change to 3 -> 1; This means this dataset is built upon Grayscale

- **Also in model.py we changed hardcoded input_channel to generalized form which is directly uses in_channel value from config file.**

```
in_channels = kwargs.get("in_channels", 3)
```

then training:

```
Epoch [01/50] | Train Loss: 0.2739 - Train Acc: 88.34% | Val Loss: 1.2866 - Val Acc: 75.91%
Epoch [02/50] | Train Loss: 0.1553 - Train Acc: 94.00% | Val Loss: 2.4251 - Val Acc: 26.77%
Epoch [03/50] | Train Loss: 0.1243 - Train Acc: 95.74% | Val Loss: 0.5150 - Val Acc: 77.63%
Epoch [04/50] | Train Loss: 0.1032 - Train Acc: 96.39% | Val Loss: 0.2040 - Val Acc: 92.93%
Epoch [05/50] | Train Loss: 0.0921 - Train Acc: 96.83% | Val Loss: 0.5269 - Val Acc: 83.37%
Epoch [06/50] | Train Loss: 0.0939 - Train Acc: 96.85% | Val Loss: 0.1929 - Val Acc: 93.31%
Epoch [07/50] | Train Loss: 0.0782 - Train Acc: 97.36% | Val Loss: 0.1230 - Val Acc: 95.60%
Epoch [08/50] | Train Loss: 0.0751 - Train Acc: 97.32% | Val Loss: 0.0643 - Val Acc: 97.51%
Epoch [09/50] | Train Loss: 0.0732 - Train Acc: 97.32% | Val Loss: 0.0751 - Val Acc: 97.13%
Epoch [10/50] | Train Loss: 0.0636 - Train Acc: 97.94% | Val Loss: 0.0453 - Val Acc: 98.66%
Epoch [11/50] | Train Loss: 0.0503 - Train Acc: 98.20% | Val Loss: 0.2300 - Val Acc: 90.82%
Epoch [12/50] | Train Loss: 0.0429 - Train Acc: 98.41% | Val Loss: 0.0564 - Val Acc: 97.90%
```

```
{
    "DATA": "chest.pt",
    "DATA_PATH": "./Data/",
    "BATCH_SIZE": 32,
    "LEARNING_RATE": 0.001,
    "EPOCHS": 50,
    "MODEL": "ResNet18",
    "CHANNELS": 1,
    "NUM_CLASSES": 2
}
```

3. Now same **AlexNet** model but Dataset: **lesions.pt** change

About data:

Image shape: torch.Size([8010, 3, 64, 64])

Unique labels: tensor([0, 1, 2, 3, 4, 5, 6])

Num classes: 7

Changes the config file:

```
{
    "DATA": "lesions.pt",
    "DATA_PATH": "./Data/",
    "BATCH_SIZE": 32,
    "LEARNING_RATE": 0.001,
    "EPOCHS": 50,
    "MODEL": "AlexNet",
    "CHANNELS": 3,
    "NUM_CLASSES": 7
}
```

Error: ValueError: Expected input batch_size (1) to match target batch_size (0). Fit.py

Before:

```
labels = labels.squeeze()
```

After:

```
labels = labels.squeeze(1)
```

Change to both training loop as well as eval mode

```
# Normal batch (batch_size=32)
```

```
labels_normal = torch.tensor([[0], [1], [2], [0], ...]) # shape [32, 1]
```

```
print("Normal batch shape:", labels_normal.shape)
```

```
print("After .squeeze():", labels_normal.squeeze().shape) # [32] ✓
```

```
print("After .squeeze(1):", labels_normal.squeeze(1).shape) # [32] ✓
```

```
print("\n---\n")
```

```
# SMALL batch (batch_size=1) - THIS IS THE PROBLEM
```

```
labels_small = torch.tensor([[0]]) # shape [1, 1]
```

```
print("Small batch shape:", labels_small.shape)
```

```
print("After .squeeze():", labels_small.squeeze().shape) # [] ← SCALAR! EMPTY!
```

```
print("After .squeeze(1):", labels_small.squeeze(1).shape) # [1] ✓
```

Why does this happen on lesions data?

When you split data into batches and the total samples don't divide evenly by batch size, the **last batch is smaller**. If it's exactly 1 sample:

- Shape: [1, 1]
- `.squeeze()` removes **both** dimensions of size 1 → becomes scalar (batch_size = 0!)
- `.squeeze(1)` removes **only dimension 1** → stays [1] ✓

That's why the error says "Expected batch_size (1) to match target batch_size (0)" — the last validation batch broke!

Epoch [01/50] | Train Loss: 0.9869 - Train Acc: 67.19% | Val Loss: 0.9694 - Val Acc: 62.05%
Epoch [02/50] | Train Loss: 0.8831 - Train Acc: 68.58% | Val Loss: 0.9955 - Val Acc: 66.92%
Epoch [03/50] | Train Loss: 0.8474 - Train Acc: 69.64% | Val Loss: 0.8112 - Val Acc: 70.29%
Epoch [04/50] | Train Loss: 0.8301 - Train Acc: 70.56% | Val Loss: 0.8165 - Val Acc: 69.16%
Epoch [05/50] | Train Loss: 0.7896 - Train Acc: 71.27% | Val Loss: 0.8590 - Val Acc: 72.66%
Epoch [06/50] | Train Loss: 0.7655 - Train Acc: 71.90% | Val Loss: 0.6963 - Val Acc: 74.16%
Epoch [07/50] | Train Loss: 0.7469 - Train Acc: 72.52% | Val Loss: 0.7118 - Val Acc: 74.28%
Epoch [08/50] | Train Loss: 0.7390 - Train Acc: 72.73% | Val Loss: 0.6985 - Val Acc: 74.16%
Epoch [09/50] | Train Loss: 0.7288 - Train Acc: 73.42% | Val Loss: 0.7203 - Val Acc: 73.53%
Epoch [10/50] | Train Loss: 0.7023 - Train Acc: 74.16% | Val Loss: 0.6868 - Val Acc: 75.03%

We can improve the acc. By changing the learning rate, improving epochs, using different batch size 16/32,

Model stop learning after 10 epochs but before that we reach our minimum accuracy score so it doesn't matters.

4. Now same **AlexNet** model but Dataset: **Orgs.pt** change

About data:

Image shape: torch.Size([15367, 1, 64, 64])

Unique labels: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

Num classes: 11

Config.js:

```
{  
  "DATA": "orgs.pt",  
  "DATA_PATH": "./Data/",  
  "BATCH_SIZE": 32,  
  "LEARNING_RATE": 0.001,  
  "EPOCHS": 50,  
  "MODEL": "AlexNet",  
  "CHANNELS": 1,  
  "NUM_CLASSES": 11  
}
```

Booom! No error at all.

Larger dataset:

- Lesions: 8,010 samples
- Orgs: 15,367 samples (almost 2x!)

With more data, the model gets better gradient estimates per batch, so it learns more efficiently.

2. Data might be more separable:

Different medical image types have different visual characteristics. Orgs images might be **more distinct between classes** than lesions, making them easier to classify.

3. No errors!

This is the key — we've already fixed all the bugs before this run:

-  Dropout rate fixed
-  Missing zero_grad() fixed
-  Labels squeeze fixed
-  in_channels made flexible
-  All config values correct

Epoch [01/50] | Train Loss: 0.7950 - Train Acc: 72.61% | Val Loss: 0.4368 - Val Acc: 88.35%

Epoch [02/50] | Train Loss: 0.3952 - Train Acc: 87.65% | Val Loss: 0.1205 - Val Acc: 96.48%

Epoch [03/50] | Train Loss: 0.2853 - Train Acc: 90.56% | Val Loss: 0.1310 - Val Acc: 95.77%

Epoch [04/50] | Train Loss: 0.2713 - Train Acc: 91.40% | Val Loss: 0.1128 - Val Acc: 96.74%

Epoch [05/50] | Train Loss: 0.2203 - Train Acc: 92.94% | Val Loss: 0.2369 - Val Acc: 94.21%

Epoch [06/50] | Train Loss: 0.2056 - Train Acc: 93.30% | Val Loss: 0.0330 - Val Acc: 99.15%

Epoch [07/50] | Train Loss: 0.1721 - Train Acc: 94.26% | Val Loss: 0.0568 - Val Acc: 97.98%

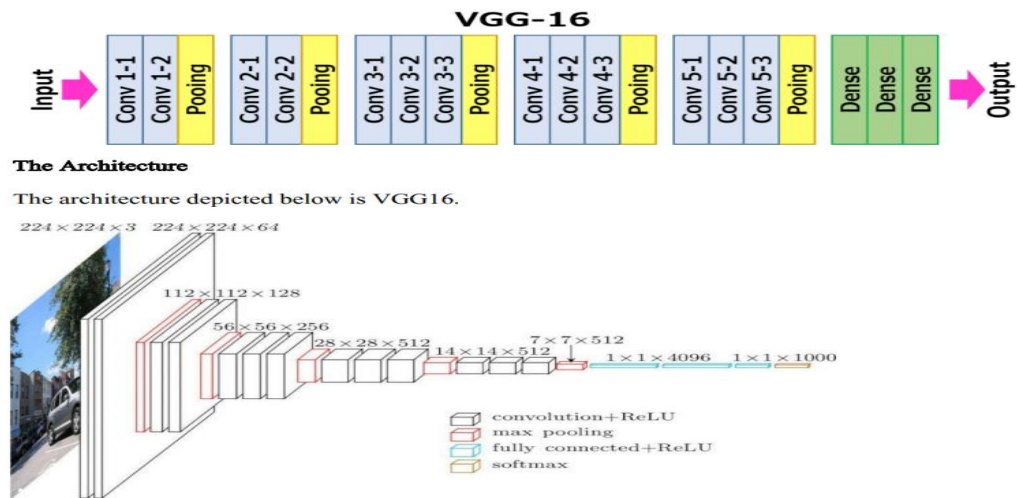
Epoch [08/50] | Train Loss: 0.1775 - Train Acc: 94.23% | Val Loss: 0.0339 - Val Acc: 98.63%

Epoch [09/50] | Train Loss: 0.1642 - Train Acc: 94.76% | Val Loss: 0.0436 - Val Acc: 98.63%

Epoch [10/50] | Train Loss: 0.1665 - Train Acc: 94.93% | Val Loss: 0.0088 - Val Acc: 99.87%

Epoch [11/50] | Train Loss: 0.1582 - Train Acc: 95.06% | Val Loss: 0.1110 - Val Acc: 95.90%

Now VGG16



VGG16 is a convolutional neural network (CNN) architecture known for its simplicity and effectiveness in image classification tasks, consisting of 16 layers with weights.

Overview of VGG16

VGG16, developed by the Visual Geometry Group at the University of Oxford, is a deep learning model designed for image recognition and classification. It is characterized by its depth and uniform architecture, which utilizes small convolutional filters to capture fine details in images. The model consists of 16 layers with learnable parameters, including 13 convolutional layers and 3 fully

connected layers  [GeeksforGeeks+1](https://www.geeksforgeeks.org/vgg-16-convolutional-neural-network/).

Architecture Details

- Convolutional Layers:** VGG16 employs a series of convolutional layers with 3x3 filters and a stride of 1, which allows it to learn complex features from the input images. The use of small filters helps in capturing more detailed patterns while maintaining a manageable number of parameters  [Built In+1](https://builtin.com/machine-learning/vgg-16).
- Max Pooling Layers:** After certain convolutional layers, VGG16 uses 2x2 max pooling layers with a stride of 2 to reduce the spatial dimensions of the feature maps. This downsampling helps in reducing computational load and extracting dominant features  [www.thinkingstack.ai](https://www.thinkingstack.ai/vgg-16-architecture/).
- Fully Connected Layers:** At the end of the network, VGG16 has three fully connected layers. The first two layers have 4096 neurons each, and the final layer uses a softmax

activation function to output probabilities for 1000 classes, corresponding to the categories in the ImageNet dataset  [GeeksforGeeks+1](#).

4. **Activation Function:** The ReLU (Rectified Linear Unit) activation function is applied after each convolutional layer to introduce non-linearity into the model, allowing it to learn more complex functions  www.thinkingstack.ai.

Performance and Applications

VGG16 has achieved impressive results in various computer vision tasks, particularly in image classification, where it attained a top-5 accuracy of 92.7% on the ImageNet dataset, which contains over 14 million images  [GeeksforGeeks+1](#). Its architecture has made it a popular choice for transfer learning, where pre-trained models are fine-tuned for specific tasks, significantly reducing the time and resources needed for training  codezup.com.

Conclusion

VGG16 remains a foundational model in the field of deep learning and computer vision, demonstrating the effectiveness of deep architectures with small convolutional filters. Its simplicity and strong performance continue to influence the design of modern CNN architectures.

1. VGG16 on cells.pt dataset

Dataset:

Image shape: torch.Size([13671, 3, 64, 64])
Unique labels: tensor([0, 1, 2, 3, 4, 5, 6, 7])
Num classes: 8

Config:

```
{  
    "DATA": "cells.pt",  
    "DATA_PATH": "./Data/",  
    "BATCH_SIZE": 32,  
    "LEARNING_RATE": 0.001,  
    "EPOCHS": 50,  
    "MODEL": "VGG16",  
    "CHANNELS": 3,  
    "NUM_CLASSES": 8  
}
```

Error: RuntimeError: Given groups=1, weight of size [64, 3, 3, 3], expected input[32, 64, 64, 64] to have 3 channels, but got 64 channels instead Model.py

Reason:

```
def __init__(self, in_channels, out_channels, num_convs, padding=1):
    super().__init__()
    layers = []
    current_in_channels = in_channels
    for i in range(num_convs):
        is_config_c_tail = (num_convs == 3 and i == 2)
        kernel_size = 1 if is_config_c_tail else 3
        layers.append(nn.Conv2d(current_in_channels, out_channels, kernel_size=kernel_size,
padding=padding))
        layers.append(nn.BatchNorm2d(out_channels))
        layers.append(nn.ReLU(inplace=True))
        current_in_channels = out_channels # Line added

    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    self.block = nn.Sequential(*layers)
```

Iteration 0 (i=0):

current_in_channels = 64 (initial value)

Conv2d(64, 128, ...) ← Takes 64 in, outputs 128

Iteration 1 (i=1):

current_in_channels = 64 ← Still 64! Never updated!

Conv2d(64, 128, ...) ← WRONG! Should take 128 in, not 64!

But the output of first Conv2d is 128 channels, so second Conv2d should receive 128, not 64!

Error: RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x4608 and 2048x1024)
Model.py

Good question! Let me trace through the transformation: from where 4608 came
Original data: [13671, 3, 64, 64]

- 13,671 = total training samples
- 3 = channels (RGB)
- 64 = height
- 64 = width

DataLoader batching:

Your config.json has BATCH_SIZE: 32. The DataLoader groups 32 samples together at a time:

Batch 1: samples 0-31 → shape [32, 3, 64, 64]

Batch 2: samples 32-63 → shape [32, 3, 64, 64]

Batch 3: samples 64-95 → shape [32, 3, 64, 64]

...

Total batches: 13671 / 32 ≈ 427 batches

During forward pass:

Each batch of [32, 3, 64, 64] goes through VGG16.features:

Input: [32, 3, 64, 64]

↓ VGGBlock(3, 64, 2) → [32, 64, 32, 32]

↓ VGGBlock(64, 128, 2) → [32, 128, 16, 16]

↓ VGGBlock(128, 256, 3) → [32, 256, 8, 8]

↓ VGGBlock(256, 512, 3) → [32, 512, 4, 4]

↓ VGGBlock(512, 512, 3) → [32, 512, 3, 3] ← YOUR OUTPUT

Each MaxPool2d shrinks spatial dimensions by 2, and each Conv2d layer increases channels.

Result: [32, 512, 3, 3]

- 32 = batch_size (unchanged)
- 512 = channels (final from last VGGBlock)
- 3×3 = spatial dimensions (shrunk from 64×64)

Before:

```
self.classifier = nn.Sequential(
    nn.Linear(2048, 1024), # 2048 causing problem need to change 4608 either by hardcoded or dynamically
    nn.ReLU(inplace=True),
    nn.Dropout(p=drop_rate),
    nn.Linear(1024, 512),
    nn.ReLU(inplace=True),
    nn.Dropout(p=drop_rate),
    nn.Linear(512, num_classes))
```

Sol. Dynamic input size

Calculate flattened size dynamically

```
dummy_input = torch.randn(1, in_channels, 64, 64)
dummy_output = self.features(dummy_input)
flattened_size = dummy_output.view(1, -1).shape[1]
# dummy_output [1, 512, 3, 3]
# → .view(1, -1) → [1, 4608]
# → .shape → (1, 4608)
# → [1] → 4608
```

2. VGG16 on chest.pt dataset

Dataset:

Image shape: torch.Size([5232, 1, 64, 64])

Unique labels: tensor([0, 1])

Num classes: 2

Config:

I saw the Dataset so need to change config.js file only 1 channel, num_classes 2

```
{
  "DATA": "chest.pt",
  "DATA_PATH": "./Data/",
  "BATCH_SIZE": 32,
  "LEARNING_RATE": 0.001,
  "EPOCHS": 50,
  "MODEL": "VGG16",
```

```
"CHANNELS": 1,  
"NUM_CLASSES": 2  
}  
Training executing on device: cuda  
Data/chest.pt  
5232
```

Starting Training Routine...

```
-----  
Epoch [01/50] | Train Loss: 0.3208 - Train Acc: 87.69% | Val Loss: 1.0342 - Val Acc: 61.57%  
Epoch [02/50] | Train Loss: 0.1773 - Train Acc: 93.75% | Val Loss: 0.7931 - Val Acc: 82.41%  
Epoch [03/50] | Train Loss: 0.1756 - Train Acc: 93.64% | Val Loss: 0.1766 - Val Acc: 94.46%  
Epoch [04/50] | Train Loss: 0.1400 - Train Acc: 95.05% | Val Loss: 0.1135 - Val Acc: 96.18%  
Epoch [05/50] | Train Loss: 0.1222 - Train Acc: 95.41% | Val Loss: 0.1238 - Val Acc: 95.60%  
Epoch [06/50] | Train Loss: 0.1046 - Train Acc: 96.50% | Val Loss: 0.0715 - Val Acc: 97.13%  
Epoch [07/50] | Train Loss: 0.1093 - Train Acc: 96.02% | Val Loss: 0.1037 - Val Acc: 96.75%  
Epoch [08/50] | Train Loss: 0.0894 - Train Acc: 96.75% | Val Loss: 0.0808 - Val Acc: 96.94%  
Epoch [09/50] | Train Loss: 0.0887 - Train Acc: 96.75% | Val Loss: 0.1195 - Val Acc: 97.32%  
Epoch [10/50] | Train Loss: 0.0942 - Train Acc: 96.98% | Val Loss: 0.1769 - Val Acc: 92.93%  
Epoch [11/50] | Train Loss: 0.0858 - Train Acc: 96.79% | Val Loss: 0.0693 - Val Acc: 97.51%
```

3. VGG16 on lesions.pt dataset

Dataset:

Image shape: torch.Size([8010, 3, 64, 64])
Unique labels: tensor([0, 1, 2, 3, 4, 5, 6])
Num classes: 7

Config:

```
{  
  "DATA": "lesions.pt",  
  "DATA_PATH": "./Data/",  
  "BATCH_SIZE": 32,  
  "LEARNING_RATE": 0.001,  
  "EPOCHS": 50,  
  "MODEL": "VGG16",  
  "CHANNELS": 3,  
  "NUM_CLASSES": 7  
}
```

Epoch [01/50] | Train Loss: 1.1163 - Train Acc: 66.34% | Val Loss: 1.0430 - Val Acc: 65.92%
Epoch [02/50] | Train Loss: 1.0177 - Train Acc: 66.69% | Val Loss: 1.0681 - Val Acc: 65.92%
Epoch [03/50] | Train Loss: 0.9792 - Train Acc: 66.84% | Val Loss: 0.9583 - Val Acc: 65.92%
Epoch [04/50] | Train Loss: 0.9388 - Train Acc: 66.79% | Val Loss: 0.9567 - Val Acc: 65.92%
Epoch [05/50] | Train Loss: 0.9204 - Train Acc: 66.95% | Val Loss: 0.9072 - Val Acc: 64.42%
.
.
.
.
Epoch [15/50] | Train Loss: 0.8255 - Train Acc: 69.35% | Val Loss: 0.8062 - Val Acc: 70.79%
Epoch [16/50] | Train Loss: 0.8174 - Train Acc: 69.85% | Val Loss: 0.8563 - Val Acc: 67.04%
Epoch [17/50] | Train Loss: 0.8016 - Train Acc: 70.24% | Val Loss: 0.7768 - Val Acc: 73.03%
Epoch [18/50] | Train Loss: 0.7927 - Train Acc: 70.57% | Val Loss: 0.7590 - Val Acc: 70.79%
Epoch [19/50] | Train Loss: 0.7931 - Train Acc: 70.92% | Val Loss: 0.7901 - Val Acc: 69.66%

By the end of **50 Epoch 83% Acc.**

4. VGG16 on orgs.pt dataset

Dataset:

Image shape: torch.Size([500, 1, 64, 64])

Unique labels: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

Num classes: 11

Config:

```
{  
  "DATA": "orgs.pt",  
  "DATA_PATH": "./Data/",  
  "BATCH_SIZE": 32,  
  "LEARNING_RATE": 0.001,  
  "EPOCHS": 50,  
  "MODEL": "VGG16",  
  "CHANNELS": 1,  
  "NUM_CLASSES": 11  
}
```

Epoch [01/50] | Train Loss: 1.4257 - Train Acc: 49.02% | Val Loss: 0.7012 - Val Acc: 70.57%

Epoch [02/50] | Train Loss: 0.7174 - Train Acc: 73.09% | Val Loss: 0.2606 - Val Acc: 87.63%

Epoch [03/50] | Train Loss: 0.5046 - Train Acc: 80.76% | Val Loss: 0.2271 - Val Acc: 91.34%

Epoch [04/50] | Train Loss: 0.4324 - Train Acc: 83.95% | Val Loss: 0.2489 - Val Acc: 90.69%

Epoch [05/50] | Train Loss: 0.3499 - Train Acc: 87.91% | Val Loss: 0.1300 - Val Acc: 96.16%

Epoch [06/50] | Train Loss: 0.2887 - Train Acc: 90.66% | Val Loss: 0.1507 - Val Acc: 95.70%

Epoch [07/50] | Train Loss: 0.2290 - Train Acc: 92.59% | Val Loss: 0.0598 - Val Acc: 98.24%

Epoch [08/50] | Train Loss: 0.2329 - Train Acc: 92.68% | Val Loss: 0.3178 - Val Acc: 93.42%

Epoch [09/50] | Train Loss: 0.2950 - Train Acc: 90.96% | Val Loss: 0.0895 - Val Acc: 97.07%

Epoch [10/50] | Train Loss: 0.1899 - Train Acc: 93.86% | Val Loss: 0.0543 - Val Acc: 98.24%

Epoch [11/50] | Train Loss: 0.1919 - Train Acc: 94.16% | Val Loss: 0.0380 - Val Acc: 98.83%

Epoch [12/50] | Train Loss: 0.1603 - Train Acc: 94.91% | Val Loss: 0.0141 - Val Acc: 99.41%

Epoch [13/50] | Train Loss: 0.1589 - Train Acc: 95.05% | Val Loss: 0.0567 - Val Acc: 98.44%

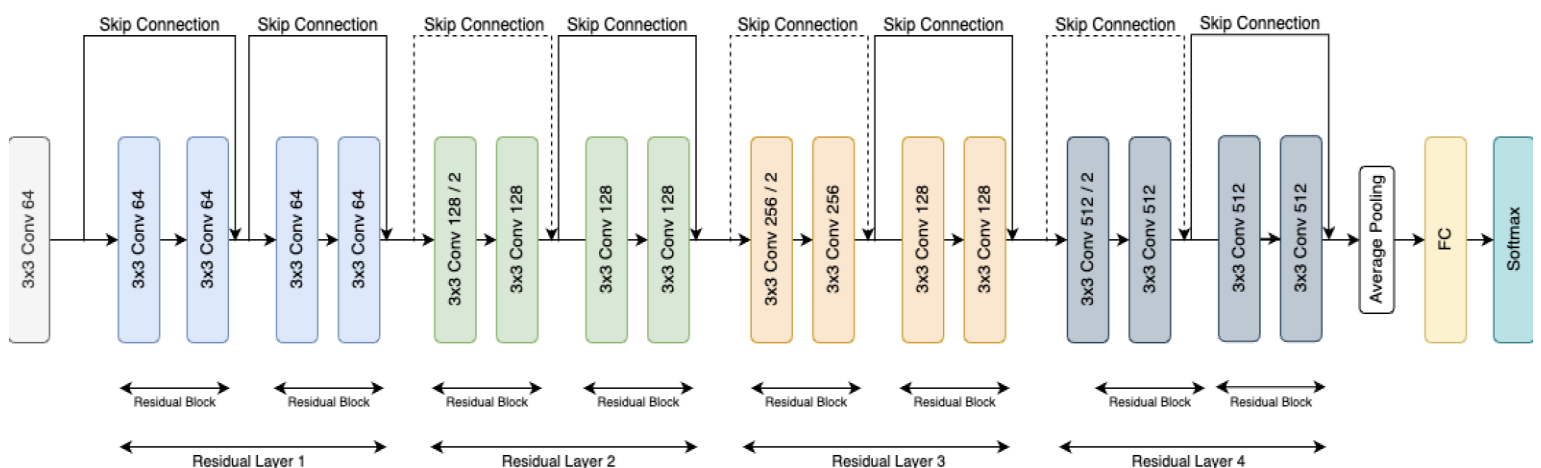
Epoch [14/50] | Train Loss: 0.1338 - Train Acc: 95.72% | Val Loss: 0.0107 - Val Acc: 99.48%

Epoch [15/50] | Train Loss: 0.1331 - Train Acc: 95.85% | Val Loss: 0.0390 - Val Acc: 98.11%

Epoch [16/50] | Train Loss: 0.1309 - Train Acc: 95.85% | Val Loss: 0.0051 - Val Acc: 99.93%

Now ResNet18

Original ResNet 18



ResNet18 = Residual Network

The key difference from VGG/AlexNet:

In VGG, each layer transforms the input:

$x \rightarrow \text{Conv} \rightarrow x'$ (old x is thrown away)

In ResNet, each layer has a **skip connection** (shortcut):

$x \rightarrow \text{Conv} \rightarrow x'$

↓ _____ ↑

$x' = x' + x$ (ADD the original input back!)

This is called a **residual block**. Instead of learning $x \rightarrow x'$, the network learns $x \rightarrow (x' - x)$, which is the **residual** (the difference).

Why is this better?

1. **Easier to train deep networks** — Gradients flow directly through skip connections
 2. **Vanishing gradient problem solved** — Deep networks don't lose signal
 3. **Better accuracy** — Can go much deeper (ResNet has 50, 101, 152 layer versions)
-

Part 1: Why ResNet is Better

1. Easier Gradient Flow Through Skip Connections

Without skip connections (regular network):

Forward: $x \rightarrow \text{Conv1} \rightarrow \text{Conv2} \rightarrow \text{Conv3} \rightarrow \text{output}$

Backward: gradient flows back through $\text{Conv3} \rightarrow \text{Conv2} \rightarrow \text{Conv1}$

At each layer, gradient gets multiplied by weights

If weight < 1 : gradient shrinks exponentially

If weight > 1 : gradient explodes exponentially

With skip connections (ResNet):

Forward: $x \rightarrow \text{Conv1} \rightarrow \text{Conv2} \rightarrow \text{output}$

↓ _____ ↑

Identity (direct path)

Backward: gradient can flow DIRECTLY through skip connection

+ also through Conv layers

Result: gradient has TWO PATHS to flow back!

The skip connection provides a "**superhighway**" for gradients to travel through the network.

2. Vanishing Gradient Problem

What is vanishing gradient?

Imagine a deep network with 50 layers:

$$\partial \text{loss} / \partial x = \partial \text{loss} / \partial \text{out} \times \partial \text{out} / \partial \text{layer50} \times \partial \text{layer50} / \partial \text{layer49} \times \dots \times \partial \text{layer2} / \partial \text{layer1} \times \partial \text{layer1} / \partial x$$

This is a PRODUCT of 50 terms, each typically < 1

Result: $0.9^{50} \approx 0.005$ (gradient becomes nearly 0!)

With ResNet's skip connections:

Gradient = gradient_through_skip + gradient_through_convs

= 1 + (some small value)

≈ 1 (doesn't vanish!)

The skip connection adds a constant gradient of **1**, so deep layers still receive meaningful gradients.

3. Can Go Much Deeper

Because gradients don't vanish, you can stack many blocks:

- **AlexNet**: 8 layers max, or gradients die
- **VGG16**: 16 layers, still hits limits
- **ResNet**: 50, 101, 152, even 1000+ layers! (because skip connections preserve gradients)

Deeper = more parameters = better pattern recognition.

Part 2: Line-by-Line Code Explanation

```
def forward(self, x):  
    identity = self.shortcut(x)  
    out = self.activation(self.bn1(self.conv1(x)))  
    out = self.bn2(self.conv2(out))  
    out += identity  
    out = self.activation(out)  
    return out
```

Let me trace through with concrete example:

Assume:

- Input x shape: [32, 64, 32, 32] (batch=32, channels=64, spatial=32×32)
 - ResBlock config: in_channels=64, out_channels=128, stride=2
-

Line 1: `identity = self.shortcut(x)`

What is shortcut?

If stride=2 or channels change (64→128), we need to transform the input to match the output shape for the skip connection.

In `__init__`:

```
if stride != 1 or in_channels != out_channels:  
    self.shortcut = Conv2d(64, 128, kernel=1, stride=2)  
else:  
    self.shortcut = Identity() (no transformation)
```

Execution:

```
identity = shortcut(x)  
identity shape: [32, 128, 16, 16] (channels increased, spatial halved)
```

Pseudocode:

identity = x, but adjusted to match output format

(If input is 64 channels and output will be 128,
identity gets transformed to 128 channels too)

Line 2: out = self.activation(self.bn1(self.conv1(x)))

Step 1: self.conv1(x)

Conv2d(in_channels=64, out_channels=128, kernel=3, stride=2, padding=1)

Input: [32, 64, 32, 32]

Output: [32, 128, 16, 16]

Step 2: self.bn1(...) (BatchNorm)

Normalizes the output of Conv1

Shape stays: [32, 128, 16, 16]

(Makes training more stable)

Step 3: self.activation(...) (ReLU)

Applies ReLU activation

Output: [32, 128, 16, 16]

Pseudocode:

out = ReLU(BatchNorm(Conv1(x)))

Convert 64-channel image to 128-channel, shrink spatial size by 2

Normalize the result

Apply ReLU to activate

Line 3: out = self.bn2(self.conv2(out))

Step 1: self.conv2(out)

Conv2d(in_channels=128, out_channels=128, kernel=3, stride=1, padding=1)

Input: [32, 128, 16, 16]

Output: [32, 128, 16, 16] (same shape, just refines features)

Step 2: self.bn2(...)

Batch normalization

Output: [32, 128, 16, 16]

NOTE: No ReLU here yet!

Pseudocode:

out = BatchNorm(Conv2(out))

Apply another convolution (no activation yet)

This learns deeper feature transformations

Line 4: out += identity

This is THE KEY RESIDUAL CONNECTION!

Before: out shape [32, 128, 16, 16]
identity shape [32, 128, 16, 16]

Operation: Element-wise addition

out = out + identity
[32, 128, 16, 16] + [32, 128, 16, 16] = [32, 128, 16, 16]

Mathematically:

out = Conv2(BN(Conv1(x))) + shortcut(x)

This allows the network to learn:

"What's the DIFFERENCE between the input and output?"

Instead of:

"What's the output?"

Pseudocode:

output_with_skip = (result_from_convolution) + (original_input_adjusted)

If convolutions learn something useful, add it to the original

If convolutions learn nothing, original still flows through!

Line 5: out = self.activation(out)

out = ReLU(out)

Apply final activation after skip connection addition

Output: [32, 128, 16, 16]

Pseudocode:

final_output = ReLU(output_with_skip)

Apply ReLU after the skip connection merge

Resnet18 on cells.pt

Dataset:

Image shape: torch.Size([13671, 3, 64, 64])

Unique labels: tensor([0, 1, 2, 3, 4, 5, 6, 7])

Num classes: 8

Config.js

```
{  
  "DATA": "cells.pt",
```

```

"DATA_PATH": "./Data/",
"BATCH_SIZE": 32,
"LEARNING_RATE": 0.001,
"EPOCHS": 50,
"MODEL": "ResNet18",
"CHANNELS": 3,
"NUM_CLASSES": 8
}

```

1. **Error: TypeError: cross_entropy_loss(): argument 'input' (position 1) must be Tensor, not NoneType**

➔ **Sol.** There is “Return” word is missing so forward method is passing nothing and that’s why error accured.

2.

```

3. Epoch [01/50] | Train Loss: 1.1469 - Train Acc: 57.44% | Val Loss: 0.9606 - Val Acc: 62.55%
4. Epoch [02/50] | Train Loss: 0.9772 - Train Acc: 64.11% | Val Loss: 1.3731 - Val Acc: 55.01%
5. Epoch [03/50] | Train Loss: 0.8623 - Train Acc: 69.06% | Val Loss: 1.2506 - Val Acc: 55.23%
6. Epoch [04/50] | Train Loss: 0.7831 - Train Acc: 71.06% | Val Loss: 0.9594 - Val Acc: 62.77%
7. Epoch [05/50] | Train Loss: 0.7061 - Train Acc: 74.66% | Val Loss: 1.3316 - Val Acc: 51.43%
8. Epoch [06/50] | Train Loss: 0.6634 - Train Acc: 76.86% | Val Loss: 0.8367 - Val Acc: 70.23%
9. Epoch [07/50] | Train Loss: 0.6209 - Train Acc: 78.21% | Val Loss: 0.7264 - Val Acc: 71.25%
10. Epoch [08/50] | Train Loss: 0.6028 - Train Acc: 78.57% | Val Loss: 0.5557 - Val Acc: 80.91%
11. Epoch [09/50] | Train Loss: 0.5830 - Train Acc: 79.72% | Val Loss: 0.5609 - Val Acc: 79.96%

```

➔ This training is without non-linear activations(activation_str = "Identity"), your network becomes **essentially linear** — just a series of matrix multiplications. Linear networks can't learn complex patterns! That's why ResNet18 is learning slower (79% at epoch 9) compared to AlexNet (90% at epoch 4).

```

Epoch [01/50] | Train Loss: 0.4342 - Train Acc: 84.07% | Val Loss: 0.7346 - Val Acc: 74.84%
Epoch [02/50] | Train Loss: 0.2297 - Train Acc: 91.95% | Val Loss: 0.3065 - Val Acc: 89.76%
Epoch [03/50] | Train Loss: 0.1863 - Train Acc: 93.39% | Val Loss: 2.4038 - Val Acc: 62.18%
Epoch [04/50] | Train Loss: 0.1606 - Train Acc: 94.62% | Val Loss: 0.1808 - Val Acc: 93.12%

```

Epoch [05/50] | Train Loss: 0.1324 - Train Acc: 95.47% | Val Loss: 0.1863 - Val Acc: 94.37%
Epoch [06/50] | Train Loss: 0.1239 - Train Acc: 95.57% | Val Loss: 0.3877 - Val Acc: 87.49%
Epoch [07/50] | Train Loss: 0.1011 - Train Acc: 96.49% | Val Loss: 0.0884 - Val Acc: 96.63%
Epoch [08/50] | Train Loss: 0.1116 - Train Acc: 96.33% | Val Loss: 0.6652 - Val Acc: 79.52%
Epoch [09/50] | Train Loss: 0.0929 - Train Acc: 96.74% | Val Loss: 0.7051 - Val Acc: 84.86%
Epoch [10/50] | Train Loss: 0.0800 - Train Acc: 97.30% | Val Loss: 0.0600 - Val Acc: 97.59%
Epoch [11/50] | Train Loss: 0.0751 - Train Acc: 97.46% | Val Loss: 0.4423 - Val Acc: 84.64%
Epoch [12/50] | Train Loss: 0.0837 - Train Acc: 97.21% | Val Loss: 0.3253 - Val Acc: 91.37%
Epoch [13/50] | Train Loss: 0.0683 - Train Acc: 97.59% | Val Loss: 0.5917 - Val Acc: 83.17%
Epoch [14/50] | Train Loss: 0.0644 - Train Acc: 97.77% | Val Loss: 0.1539 - Val Acc: 94.22%
Epoch [15/50] | Train Loss: 0.0583 - Train Acc: 97.95% | Val Loss: 0.0308 - Val Acc: 98.98%

ResNet18 on chest.py

After changing config file there is not error;

Epoch [01/50] | Train Loss: 0.2675 - Train Acc: 89.87% | Val Loss: 0.2838 - Val Acc: 88.34%
Epoch [02/50] | Train Loss: 0.1645 - Train Acc: 93.52% | Val Loss: 0.2814 - Val Acc: 90.82%
Epoch [03/50] | Train Loss: 0.1354 - Train Acc: 94.71% | Val Loss: 0.2175 - Val Acc: 91.78%
Epoch [04/50] | Train Loss: 0.1431 - Train Acc: 94.34% | Val Loss: 0.1905 - Val Acc: 92.54%
Epoch [05/50] | Train Loss: 0.1083 - Train Acc: 96.08% | Val Loss: 0.0902 - Val Acc: 96.37%
Epoch [06/50] | Train Loss: 0.0973 - Train Acc: 96.41% | Val Loss: 0.4603 - Val Acc: 84.13%
Epoch [07/50] | Train Loss: 0.0876 - Train Acc: 96.67% | Val Loss: 0.1825 - Val Acc: 92.73%
Epoch [08/50] | Train Loss: 0.0722 - Train Acc: 97.23% | Val Loss: 0.3311 - Val Acc: 87.19%
Epoch [09/50] | Train Loss: 0.0820 - Train Acc: 97.02% | Val Loss: 0.1252 - Val Acc: 95.22%
Epoch [10/50] | Train Loss: 0.0711 - Train Acc: 97.38% | Val Loss: 0.1083 - Val Acc: 95.41%
Epoch [11/50] | Train Loss: 0.0694 - Train Acc: 97.32% | Val Loss: 0.0367 - Val Acc: 98.85%
Epoch [12/50] | Train Loss: 0.0551 - Train Acc: 97.92% | Val Loss: 0.0807 - Val Acc: 96.75%
Epoch [13/50] | Train Loss: 0.0626 - Train Acc: 97.52% | Val Loss: 0.0485 - Val Acc: 98.28%

Resnet18 on lesions.pt

After changing config file there is not error;

Epoch [01/50] | Train Loss: 0.9540 - Train Acc: 66.82% | Val Loss: 0.9376 - Val Acc: 65.29%
Epoch [02/50] | Train Loss: 0.8738 - Train Acc: 68.20% | Val Loss: 0.8590 - Val Acc: 68.16%
Epoch [03/50] | Train Loss: 0.8357 - Train Acc: 69.33% | Val Loss: 0.9349 - Val Acc: 64.04%
Epoch [04/50] | Train Loss: 0.8147 - Train Acc: 70.35% | Val Loss: 1.0205 - Val Acc: 60.42%
Epoch [05/50] | Train Loss: 0.7859 - Train Acc: 70.46% | Val Loss: 0.9450 - Val Acc: 58.80%
Epoch [06/50] | Train Loss: 0.7637 - Train Acc: 71.99% | Val Loss: 0.8291 - Val Acc: 70.41%
Epoch [07/50] | Train Loss: 0.7253 - Train Acc: 73.31% | Val Loss: 0.8541 - Val Acc: 69.29%
Epoch [08/50] | Train Loss: 0.7174 - Train Acc: 73.30% | Val Loss: 0.7423 - Val Acc: 74.66%
Epoch [09/50] | Train Loss: 0.7062 - Train Acc: 73.87% | Val Loss: 0.7001 - Val Acc: 73.78%
Epoch [10/50] | Train Loss: 0.6814 - Train Acc: 74.51% | Val Loss: 0.6374 - Val Acc: 76.15%

Resnet18 on lesions.pt

After changing config file there is not error;

Epoch [01/50] | Train Loss: 0.5961 - Train Acc: 78.54% | Val Loss: 0.1667 - Val Acc: 93.62%
Epoch [02/50] | Train Loss: 0.2609 - Train Acc: 90.71% | Val Loss: 0.2912 - Val Acc: 91.28%
Epoch [03/50] | Train Loss: 0.1857 - Train Acc: 93.57% | Val Loss: 0.0250 - Val Acc: 99.54%
Epoch [04/50] | Train Loss: 0.1497 - Train Acc: 94.65% | Val Loss: 0.1749 - Val Acc: 92.38%
Epoch [05/50] | Train Loss: 0.1163 - Train Acc: 95.71% | Val Loss: 0.0521 - Val Acc: 98.18%